
OPENCV DESDE PYTHON:

CALIBRACIÓN DEL PAR DE ESTEREOVISIÓN

INTRODUCCIÓN

En el segundo proyecto de la asignatura se utilizarán dos cámaras para posicionar al menos un objeto real mediante estereovisión. Para ello, en primer lugar, se deberán calibrar las dos cámaras del par estéreo utilizando la funcionalidad de OpenCV. Es importante destacar que las dos cámaras deben de estar calibradas con respecto al mismo sistema de coordenadas del mundo.

Existen muchos métodos que permiten estimar las matrices de proyección de las dos cámaras. Por una parte, se puede calibrar las cámaras aplicando el método de calibración individual a cada una de ellas utilizando el mismo sistema de coordenadas del mundo (método a). Por otra parte, si se dispone de la calibración individual de las cámaras con respecto a distintos sistemas de mundo, se podrá calibrar la localización relativa entre las dos cámaras del par de estereovisión directamente utilizando las correspondencias de píxeles homólogos con los puntos del mundo (método b) o a través de la estimación de la matriz fundamental utilizando solo con la correspondencia de píxeles homólogos (método c).

OBJETIVOS

El objetivo principal de esta práctica es seleccionar el método de calibración del par de estereovisión que se aplicará en el segundo proyecto.

MATERIALES

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, “[working_folder_path](#)”) en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/52-PyCharm demo - Camera calibration
- PoliformaT -> Recursos/2-Software-&-cuadernos/100-PyCharm demo- Stereo Calibration

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica.

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda 3)”, tras activar el “environment” adecuado.

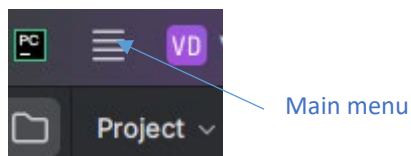
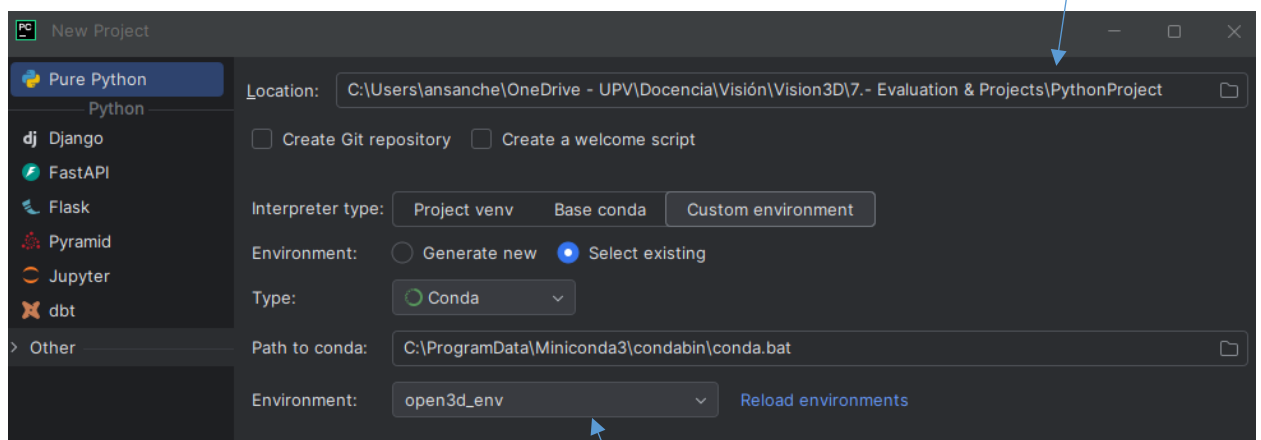
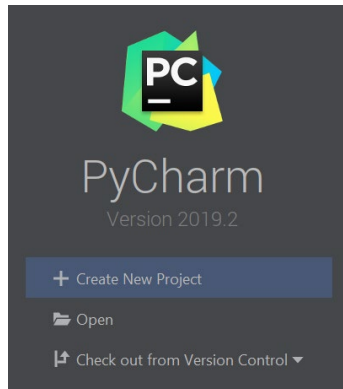
```
(base) C:\windows\system32> conda activate open3d_env
```

```
(open3d_env) C:\Users\username> jupyter lab --notebook-dir “working\_folder\_path”
```

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

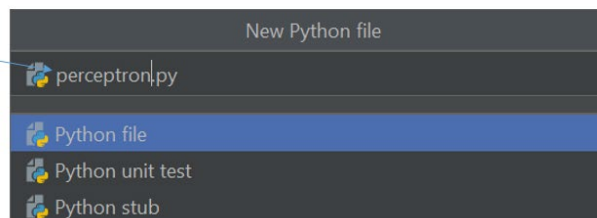
Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



DEMO 1: CALIBRACIÓN DEL PAR DE ESTEREOVISIÓN

```
import cv2 as cv
import numpy as np
import os
import matplotlib.pyplot as plt
from glob import glob

# Function to read individual camera calibration
def load_calibration(filename): ...
def load_stereo_calibration(filename):...
def get_corners(image, pattern_size):...

def add_pair_pts(pattern_size):
    objps, lettps, rightps = [], [], []

    images_left = glob("./imgs/camera_STEREO/chessboard_9x6/left_*.bmp")
    idx_left = len(images_left)

    images_right = glob("./imgs/camera_STEREO/chessboard_9x6/right_*.bmp")
    idx_right = len(images_right)

    if idx_left == idx_right:

        objp = np.zeros((1, 6 * 9, 3), np.float32)
        objp[0, :, :2] = np.mgrid[0:6, 0:9].T.reshape(-1, 2)

        for name1, name2 in zip(images_left, images_right):
            img1 = cv.imread(name1)
            ret1, leftp = get_corners(img1, pattern_size)

            img2 = cv.imread(name2)
            ret2, rightp = get_corners(img2, pattern_size)

            if ret1 and ret2:
                objps.append(objp)
                lettps.append(leftp)
                rightps.append(rightp)

    if idx_left == 0:
        img_shape = (0,0)
        print("Error: There isn't calibration images...")
    else:
        img_shape = img1.shape[:2][::-1]

    return objps, lettps, rightps, img_shape

def calibrate_stereo_cameras():

    # Load individual left camera calibration
    K_l, RT_l, dist_l = load_calibration('./imgs/camera_LEFT/chessboard_9x6/calib.npy')

    # Load individual right camera calibration
    K_r, RT_r, dist_r = load_calibration('./imgs/camera_RIGHT/chessboard_9x6/calib.npy')

    pattern_size = (9, 6)
    objps, lettps, rightps, img_shape = add_pair_pts(pattern_size)

    criteria = (cv.TERM_CRITERIA_MAX_ITER + cv.TERM_CRITERIA_EPS, 100, 1e-5)
    flags = (cv.CALIB_FIX_ASPECT_RATIO + cv.CALIB_FIX_INTRINSIC)

    ret, K1, D1, K2, D2, R, T, E, F = cv.stereoCalibrate(objps,
                                                         lettps,
                                                         rightps,
                                                         K_l,
                                                         dist_l,
                                                         K_r,
                                                         dist_r,
                                                         img_shape,
                                                         criteria=criteria,
                                                         flags=flags)

    return ret, R, T, E, F

if __name__ == "__main__":

    # Load camera calibration
    ret, R, T, E, F = load_stereo_calibration("./imgs/camera_STEREO/chessboard_9x6/stereo_calib.npy")

    # Stereo camera calibration
    ret, R, T, E, F = calibrate_stereo_cameras()
```

DEMO 2: ESTIMACIÓN DE LA MATRIZ FUNDAMENTAL

```
import numpy as np
import cv2 as cv

# Load two images
img1 = cv.imread('./data/KITTI07/image_0/000000.png')
img2 = cv.imread('./data/KITTI07/image_0/000023.png')
assert (img1 is not None) and (img2 is not None), 'Cannot read the given images'
f, cx, cy = 707.0912, 601.8873, 183.1104 # From the KITTI dataset
K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])

# Retrieve matching points
fdetector = cv.BRISK_create()
keypoints1, descriptors1 = fdetector.detectAndCompute(img1, None)
keypoints2, descriptors2 = fdetector.detectAndCompute(img2, None)

fmatcher = cv.DescriptorMatcher_create('BruteForce-Hamming')
match = fmatcher.match(descriptors1, descriptors2)

# Calculate the fundamental matrix
pts1, pts2 = [], []
for i in range(len(match)):
    pts1.append(keypoints1[match[i].queryIdx].pt)
    pts2.append(keypoints2[match[i].trainIdx].pt)
pts1 = np.array(pts1, dtype=np.float32)
pts2 = np.array(pts2, dtype=np.float32)
F, inlier_mask = cv.findFundamentalMat(pts1, pts2, cv.FM_RANSAC, 0.5, 0.999)
print(f'* F = {F}')
print(f'* The number of inliers = {sum(inlier_mask.ravel())}')

# Extract relative camera pose between two images
E = K.T @ F @ K
positive_num, R, t, positive_mask = cv.recoverPose(E, pts1, pts2, K, mask=inlier_mask)
print(f'* R = {R}')
print(f'* t = {t}')
print(f'* The position of Image #2 = {-R.T @ t}') # [-0.57, 0.09, 0.82]
print(f'* The number of positive-depth inliers = {sum(positive_mask.ravel())}')

# Show the matched images
img_matched = cv.drawMatches(img1, keypoints1, img2, keypoints2, match, None, None, None,
                             matchesMask=inlier_mask.ravel().tolist()) # Remove
`matchesMask` if you want to show all putative matches
cv.namedWindow('Fundamental Matrix Estimation', cv.WINDOW_NORMAL)
cv.imshow('Fundamental Matrix Estimation', img_matched)
cv.waitKey(0)
cv.destroyAllWindows()
```

DEMO 3: VISUALIZACIÓN DE RECTAS EPIPOLARES

```
import numpy as np
import cv2 as cv
import random

def mouse_event_handler(event, x, y, flags, param):
    if event == cv.EVENT_LBUTTONDOWN:
        param.append((x, y))

def draw_straight_line(img, line, color, thickness=1):
    assert img.ndim >= 2
    h, w, *_ = img.shape
    a, b, c = line # Line: ax + by + c = 0
    if abs(a) > abs(b):
        pt1 = (int(c / -a), 0)
        pt2 = (int((b*h + c) / -a), h)
    else:
        pt1 = (0, int(c / -b))
        pt2 = (w, int((a*w + c) / -b))
    cv.line(img, pt1, pt2, color, thickness)

if __name__ == '__main__':
    # Load two images
    img1 = cv.imread('./data/KITTI07/image_0/000000.png', cv.IMREAD_COLOR)
    img2 = cv.imread('./data/KITTI07/image_0/000023.png', cv.IMREAD_COLOR)
    assert (img1 is not None) and (img2 is not None), 'Cannot read the given images'
    # Note) `F` is derived from `fundamental_mat_estimation.py`.
    F = np.array([[ 3.34638533e-07,  7.58547151e-06, -2.04147752e-03],
                  [-5.83765868e-06,  1.36498636e-06,  2.67566877e-04],
                  [ 1.45892349e-03, -4.37648316e-03,  1.00000000e+00]])

    # Register event handlers and show images
    wnd1_name, wnd2_name = 'Epipolar Line: Image #1', 'Epipolar Line: Image #2'
    img1_pts, img2_pts = [], []
    cv.namedWindow(wnd1_name)
    cv.namedWindow(wnd2_name)
    cv.setMouseCallback(wnd1_name, mouse_event_handler, img1_pts)
    cv.setMouseCallback(wnd2_name, mouse_event_handler, img2_pts)
    cv.imshow(wnd1_name, img1)
    cv.imshow(wnd2_name, img2)

    # Get a point from a image and draw its corresponding epipolar line on the other image
    while True:
        if len(img1_pts) > 0:
            for x, y in img1_pts:
                color = (random.randrange(256), random.randrange(256), random.randrange(256))
                cv.circle(img1, (x, y), 4, color, -1)
                epipolar_line = F @ [[x], [y], [1]]
                draw_straight_line(img2, epipolar_line, color, 2)
            img1_pts.clear()
        if len(img2_pts) > 0:
            for x, y in img2_pts:
                color = (random.randrange(256), random.randrange(256), random.randrange(256))
                cv.circle(img2, (x, y), 4, color, -1)
                epipolar_line = F.T @ [[x], [y], [1]]
                draw_straight_line(img1, epipolar_line, color, 2)
            img2_pts.clear()
        cv.imshow(wnd2_name, img2)
        cv.imshow(wnd1_name, img1)
        key = cv.waitKey(10)
        if key == 27: # ESC
            break

    cv.destroyAllWindows()
```

4.-TAREA

A partir de los materiales y ejemplos anteriores deberéis integrar en vuestro proyecto:

- un método de calibración lo más preciso posible para el par de estereovisión.